

https://github.com/SETAPESU26/52_java-escape

Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.

Deliverables:

Upload the following to your Lab repo (GitHub) under Lab-4 folder,

1. Videos (before and after)
2. Updated Code
3. Chat history exported as a doc/pdf

I'll start by cloning the repo to see what's broken and what features the README asks for.

Ran 2 commands

Ran 2 commands

The README says there's one deliberate bug plus three features. Before changing anything, I'll run the original headless and check how it actually behaves.

Ran 2 commands

Ran 2 commands

Now the engine: crumble/spring handling, the lava burst cycle, the danger meter, and the camera spawn fix.

Ran a command

Ran a command

Now I'll verify each task headlessly with automated tests, since I can't play it by hand here.

Ran 2 commands

Ran 2 commands

All pass, but one of my T1 checks was a placeholder that always passed, so it proves nothing. I'll replace it with a real below-jump test, then look at the rendered frames.

Ran a command

Ran a command

Both counts are 0, including the original code, so my detector is wrong: it tracks the rect's top instead of its bottom (feet). Fixing the test:

Ran a command

Ran a command

That's a real result: the original code teleports the player onto the platform in 24 of 170 jump scenarios, and the fixed code in 0. Now a visual check of the rendering:

Ran 3 commands

Ran 3 commands

Generation is fine; seed 3 just happened to have no springs. Using seed 0:

Read 2 files, ran a command

Read 2 files, ran a command

The spawn is fixed: the player and ground are on-screen and the meter starts empty. Packaging the results:

Ran a command, shared files

Ran a command, shared files

I fixed the collision bug and added all three features, and they pass headless tests. I couldn't play it with a real window or keyboard, so please run it once before recording your video.

I cloned the repo, read the README and went through the code. The updated project is in the zip above, which contains the changed `player.py`, `world.py` and `game_engine.py`.

What changed

- Task 1, one-way platform bug (`player.py`): the old check `rect.bottom <= p.bottom + 10` let the player snap onto a platform from below. The new check records where the feet were before moving and only lands if they were at or above the platform top (plus 2px). In a test of 170 jump scenarios, the original teleported the player on top of the platform in 24 of them. The fixed version did it in none.
- Task 2, crumbling platforms: about 20% of platforms are brownish with crack marks. Landing starts a 1-second timer. The platform shakes and glows red, then disappears from the platform list.
- Task 3, spring platforms: about 15% of platforms are yellow. Landing launches the player with `vel_y = -22`, which is about 440px of rise against about 150px for a normal jump. The platform squashes and the player stretches as the recoil animation.
- Task 4, danger meter and lava burst (`game_engine.py`):
 - A danger meter under the height text fills green to red as `lava_rise` grows, with a live lava speed readout.
 - The warning "LAVA SURGE INCOMING" flashes 1.5s before each burst.
 - Every 15 seconds the lava goes 2× faster for 3 seconds, with a red screen glow and a countdown banner.

I also fixed a bug the README doesn't mention. The player spawned at y=790 on a 640px screen with the camera at 0, so the game started with the player off-screen. I changed the starting camera position.

I chose two things beyond the spec: no two special platforms are ever adjacent, and the first and last platforms are always normal. That keeps every level beatable.

What you still need to do

1. Record the videos. Record the "before" video with the original repo, then the "after" video with my version. Use your Lab-4 folder for both.
2. Push to your own account. Fork or clone the repo, copy in the three files from the zip, and push to your personal repo. Do not open a PR against SETAPESU26.
3. Export this chat as a doc or PDF for the chat history deliverable.
4. Check the pygame version. `requirements.txt` pins `pygame==2.5.2`. If that fails to install on your Python version, `pip install pygame` works. I tested with 2.6.1.

One caveat on the lab's goal of finishing in 3–4 prompts: this took several rounds because I verified each task. If your instructor counts attempts, you may want to say you used a "build, test, fix" loop.

Lab 4 lava escape

ZIP

Lab 4 lava escape

ZIP

Player

PY

World

PY

Game engine

PY

okay how do i run this on my mac now and do the recording and all

Here's the whole flow on a Mac, in order.

1. Set up the original (for the "before" video)

Open Terminal and run:

```
bash
cd ~/Desktop
git clone https://github.com/SETAPESU26/52_lava-escape original
cd original
python3 -m venv venv
source venv/bin/activate
pip install pygame

python main.py
```

If `python3` isn't found, install Python from python.org or run `brew install python`. I used `pip install pygame` instead of `-r requirements.txt` because the pinned 2.5.2 can fail on newer Python versions.

2. Record the "before" video

1. Press `Cmd + Shift + 5`. Choose Record Selected Portion and drag a box around the game window.
2. Click Options and make sure the microphone is off.
3. Click Record, then click on the game window so it gets keyboard focus.
4. Play for about 10 seconds: move with A/D or the arrows and jump with Space. Then stop with the stop button in the menu bar, or `Cmd + Ctrl + Esc`.

What the original shows is that the player starts off-screen, so you'll see an empty screen with lava rising. That's the broken behavior. Say so in a caption or when you submit.

The video is saved to your Desktop as a `.mov` file. Rename it `before.mov`.

3. Run the fixed version

Download the zip from my previous message and unzip it. Then:

```
bash
cd ~/Downloads/Lab-4_lava-escape/Lab-4    # adjust if the path differs
python main.py
```

If you get a pygame error, run `source ~/Desktop/original/venv/bin/activate` first to reuse that environment.

4. Record the "after" video

Record the same way, and play for about 10 seconds. Try to show:

- The danger meter at the top.
- A crumbling platform: land on a brownish one with cracks and watch it shake and vanish.

- A spring platform: land on a yellow one and bounce high.
- The lava burst. It starts at 13.5s with the warning and bursts at 15s, so a 10-second clip won't catch it. Either record longer than the minimum, or tell your instructor it triggers at 15s. I'd record a separate 20-second clip to be safe.

Rename the video `after.mov`.

5. Push to your own GitHub repo

First create (or open) your personal Lab repo on GitHub. Then:

bash

```
git clone https://github.com/<your-username>/<your-lab-repo>.git
cd <your-lab-repo>
mkdir -p Lab-4
cp -r ~/Downloads/Lab-4_lava-escape/Lab-4/* Lab-4/
cp ~/Desktop/before.mov ~/Desktop/after.mov Lab-4/
git add Lab-4
git commit -m "Lab 4: Lava Escape vibe coding"
```

`git push`

If you'd rather fork the assigned repo instead, fork it on GitHub to your account. Do not click "Contribute / Open pull request", since the lab says not to PR to SETAPESU26.

6. Export the chat

Save this conversation as a PDF, for example with Cmd + P and then Save as PDF in the browser, or by copying it into a doc. Put it in `Lab-4/` too.

GitHub rejects files over 100 MB, so if a video is large, trim it in QuickTime (Edit > Trim) before committing.

If anything errors when you run it, paste the error here and I'll sort it out.

what all files do i install

You only need to install two things, and there are no other files to download.

Software to install

1. Python 3.10 or newer. Check whether you already have it:

bash

```
python3 --version
```

If you see 3.10 or higher, you're set. If not, get it from python.org/downloads (or `brew install python` if you use Homebrew).

2. pygame, the game library. It installs with one command from Terminal:

bash

```
pip install pygame
```

If that says `pip: command not found`, use `pip3 install pygame` or `python3 -m pip install pygame`.

You don't need anything else. Git is already on most Macs; if Terminal asks to install "command line developer tools" the first time you use `git`, click Install.

Files to download

Only one file: the zip from my earlier message, `Lab-4_lava-escape.zip`. It contains everything:

```
Lab-4/
├── main.py
├── requirements.txt
├── README.md
├── game/
│   ├── __init__.py
│   ├── game_engine.py
│   ├── player.py
│   └── world.py
├── screenshot_before.png
└── screenshot_after.png
```

Unzip it, open Terminal in that `Lab-4` folder, and run:

```
bash
```

```
python3 main.py
```

For the "before" video you'll also need the original code, but that comes from the `git clone` command in my last message, so there's nothing to download by hand.

Optional: a virtual environment

If you want to keep pygame separate from the rest of your system, run these before

```
pip install pygame
```

```
bash
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

You'd have to run `source venv/bin/activate` again in each new Terminal window. If this is your first time, skip it and install pygame directly.

Did you get a specific error or prompt while installing? Paste it here and I'll help.

Images hidden when shared

Your install worked. pygame 2.6.1 installed fine. The error came from running `python3 main.py` in your home folder (`~`), where there's a different `main.py` that has nothing to do with this game. It's an old script that imports `apscheduler`, so ignore that error. Always `cd` into the game folder first.

You're now in `Downloads` with a fresh venv active. That venv is empty, because pygame went into your conda `base` environment, so it needs pygame again.

Run these one at a time:

```
bash
```

```
pip install pygame
```

```
ls
```

`ls` shows what's in Downloads. Look for `Lab-4_lava-escape.zip` (or a similar name). If it's still zipped, unzip it:

```
bash
unzip Lab-4_lava-escape.zip
cd Lab-4

ls
```

You should see `main.py`, `game`, `requirements.txt` and the two screenshots. If the name differs (for example `Lab_4_lava-escape.zip`), use whatever `ls` showed. Double-clicking the zip in Finder also unzips it. Then run:

```
bash

python main.py
```

A game window should open. Click on it, then use A/D to move and Space to jump.

Notes:

- `python` and `pip` work inside the venv because it's active (you see `(venv)` in your prompt). If you open a new Terminal window, you'll need to `source ~/venv/bin/activate` again, since the venv was created in your home folder.
- If a window opens and immediately closes or you get an error, paste the output here.
- The venv folder in your home directory is harmless. You can delete it later with `rm -rf ~/venv`.

```
(venv) (base) harshkalekar@Harshs-MacBook-Air downloads % unzip
Lab-4_lava-escape.zip
cd Lab-4
ls
Archive: Lab-4_lava-escape.zip
replace Lab-4/README.md? [y]es, [n]o, [A]ll, [N]one, [r]ename:
```

That prompt means a `Lab-4` folder already exists, probably because you unzipped it earlier by double-clicking in Finder. Type a capital A and press Enter to overwrite everything with the same files.

A

The `cd Lab-4` and `ls` you pasted along with the `unzip` may have been swallowed by that prompt. Once you're back at the normal `%` prompt, run these one at a time:

```
bash
cd Lab-4
ls

python main.py
```

`ls` should show `main.py`, `game`, `requirements.txt`, `README.md` and the two screenshots. After `python main.py`, a game window should open. Click on it before pressing keys.

If you get an error like `No module named pygame`, run `pip install pygame` first, since this venv started out empty.

Images hidden when shared

The game is running on your Mac. The danger meter, lava speed readout and the yellow spring platform at the top all show up.

In this screenshot you're about to die. You're standing on the old ground platform and the lava is already at your feet. The lava starts rising the moment the game begins, so you can't idle at the bottom. Jump upward immediately and keep climbing.

Controls

- A/D or ←/→ to move, Space/W/↑ to jump.
- Press R to restart at any time. You'll probably restart a few times to get a good take.

What to look for in the "after" video

1. Yellow platform: land on it and you'll shoot up about 3x higher than a normal jump. The one at the top of your screenshot is a spring.

2. Brown platform with cracks: land on it and it shakes and glows red. After 1 second it vanishes, so keep moving.
3. Danger meter: it fills up the longer you survive. You'll see it grow even in a short clip.
4. Lava burst: the "LAVA SURGE INCOMING" warning appears at about 13.5s. At 15s the screen glows red and the lava goes 2x faster for 3 seconds. To capture this, you need to survive 15+ seconds.

Recording tips

- Press `Cmd + Shift + 5` and drag a box around just the game window, as you did in the screenshot.
- Start the recording, click the game window, press `R` for a fresh run, then play.
- Aim for a run that includes a spring bounce and a crumbling platform. If you land on a spring early, you get a big height jump that makes the lava easier to outrun.

Once you have a good take, rename the video to `after.mov`. Then repeat for the "before" version with the original repo from my earlier message. In that one the player starts invisible off-screen and the lava just rises, which is the broken behavior to show.

okay i recorded the after.mov now what

Next you need the "before" video, then push everything to GitHub.

1. Record the "before" video

Get the original code and run it:

```
bash
cd ~/Downloads
git clone https://github.com/SETAPESU26/52_lava-escape original
cd original

python main.py
```

Your venv should still be active (you'll see `(venv)` in the prompt), so pygame is already installed. If you get `No module named pygame`, run `pip install pygame` first.

Record it the same way (Cmd + Shift + 5, drag a box around the window, click Record, click the game window). What you'll see is a mostly empty screen: the player is off-screen at the start and the lava rises. Record about 10 seconds, then stop. Rename the file to `before.mov`.

If you want to show the platform bug itself, it's hard to trigger by hand because the player can't be seen. The off-screen start is the visible broken behavior, so mention it in your submission.

2. Put your files in one folder

Make a folder with everything for the submission:

```
bash
cd ~/Downloads
mkdir -p lab4_submit
cp -r Lab-4/main.py Lab-4/requirements.txt Lab-4/README.md Lab-4/game
lab4_submit/

cp ~/Desktop/before.mov ~/Desktop/after.mov lab4_submit/
```

Adjust the paths if your videos saved elsewhere. QuickTime saves recordings to the Desktop by default.

3. Export this chat

In your browser, press Cmd + P, choose Save as PDF, and save it as `chat_history.pdf` into `lab4_submit/`. Make sure you scroll through the whole conversation first so it's all loaded.

4. Push to your own GitHub repo

Create your Lab repo on GitHub if you haven't already (github.com, then New repository). Then:

```
bash
cd ~/Downloads
git clone https://github.com/<your-username>/<your-lab-repo>.git
cd <your-lab-repo>
mkdir -p Lab-4
```

```
cp -r ~/Downloads/lab4_submit/* Lab-4/
git add Lab-4
git commit -m "Lab 4: Lava Escape vibe coding"

git push
```

Replace `<your-username>` and `<your-lab-repo>` with your real names. GitHub may ask you to log in. If it asks for a password, use a personal access token instead of your account password.

If a video is over 100 MB, GitHub will reject it. 10 to 20 second screen recordings are usually well under that, but if one is too big, open it in QuickTime and use Edit > Trim.

5. Check it

Open your repo on github.com and confirm `Lab-4/` contains the code, both videos and the chat PDF. Do not open a pull request to SETAPESU26.

Tell me which step you're on if anything errors, and paste the message.

```
(venv) (base) harshkalekar@Harshs-MacBook-Air Downloads % cd ~/Downloads
git clone https://github.com/<your-username>/<your-lab-repo>.git
cd <your-lab-repo>
mkdir -p Lab-4
cp -r ~/Downloads/lab4_submit/* Lab-4/
git add Lab-4
git commit -m "Lab 4: Lava Escape vibe coding"
git push
zsh: parse error near `\\n'
(venv) (base) harshkalekar@Harshs-MacBook-Air Downloads %
```

That error is my fault for not being clear. `<your-username>` and `<your-lab-repo>` were placeholders, and you pasted them literally. The `<` and `>` characters mean something special in the terminal, so it choked. Nothing ran, so nothing is broken.

Get your real repo URL

1. Go to github.com and open your personal Lab repo. If you don't have one yet, click New repository, give it a name (for example `lab-repo`), leave everything else as default, and click Create repository.
2. On the repo page, click the green Code button and copy the HTTPS URL. It looks like:

```
https://github.com/harshkalekar/lab-repo.git
```

(That's an example. Yours will have your actual username and repo name.)

Run the commands with the real URL

Replace the URL on the first line with the one you copied, and replace `lab-repo` on the second line with your repo's name (the last part of the URL, without `.git`):

```
bash
cd ~/Downloads
git clone https://github.com/harshkalekar/lab-repo.git
cd lab-repo
mkdir -p Lab-4
cp -r ~/Downloads/lab4_submit/* Lab-4/
git add Lab-4
git commit -m "Lab 4: Lava Escape vibe coding"

git push
```

If the repo is brand new and empty, `git clone` may say "You appear to have cloned an empty repository." That's fine. Continue with the remaining lines, and if `git push` complains about the branch, run:

```
bash

git push -u origin main
```

If something asks for a login

GitHub no longer accepts your account password in the terminal. When it asks for a password, paste a personal access token instead. Create one at github.com, under Settings, then Developer settings, then Personal access tokens, then Tokens (classic), then Generate new token. Tick the `repo` box and copy the token.

Before you push

Make sure `~/Downloads/lab4_submit/` actually contains your files:

```
bash
```

```
ls ~/Downloads/lab4_submit
```

You should see `main.py`, `game`, `requirements.txt`, `README.md`, `before.mov`, `after.mov`, and `chat_history.pdf`. If any of those are missing, tell me which and I'll help you find them.

```
git commit -m "Lab 4: Lava Escape vibe coding"
```

```
git push
```

```
Cloning into 'lab-repo'...
```

```
Username for 'https://github.com': HarshKalekar
```

```
Password for 'https://HarshKalekar@github.com':
```

```
remote: Invalid username or token. Password authentication is not supported for Git operations.
```

```
fatal: Authentication failed for 'https://github.com/HarshKalekar/lab-repo.git/'
```

```
cd: no such file or directory: lab-repo
```

```
warning: adding embedded git repository: Downloads/Lab-4/original
```

```
hint: You've added another git repository inside your current repository.
```

```
hint: Clones of the outer repository will not contain the contents of
```

```
hint: the embedded repository and will not know how to obtain it.
```

```
hint: If you meant to add a submodule, use:
```

```
hint:
```

```
hint:   git submodule add <url> Downloads/Lab-4/original
```

```
hint:
```

```
hint: If you added this path by mistake, you can remove it from the
```

```
hint: index with:
```

```
hint:
```

```
hint:   git rm --cached Downloads/Lab-4/original
```

```
hint:
```

```
hint: See "git help submodule" for more information.
```

```
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
```

```
[main ecef256] Lab 4: Lava Escape vibe coding
```

```
12 files changed, 446 insertions(+)
```

```
create mode 100644 Downloads/Lab-4/README.md
create mode 100644 Downloads/Lab-4/after.mov
create mode 100644 Downloads/Lab-4/before.mov
create mode 100644 Downloads/Lab-4/game/__init__.py
create mode 100644 Downloads/Lab-4/game/game_engine.py
create mode 100644 Downloads/Lab-4/game/player.py
create mode 100644 Downloads/Lab-4/game/world.py
create mode 100644 Downloads/Lab-4/main.py
create mode 160000 Downloads/Lab-4/original
create mode 100644 Downloads/Lab-4/requirements.txt
create mode 100644 Downloads/Lab-4/screenshot_after.png
create mode 100644 Downloads/Lab-4/screenshot_before.png
```

Username for <https://github.com>:

Stop that prompt first, because it's trying to push something you don't want pushed.
Press Ctrl + C.

What went wrong

1. The clone failed. `lab-repo` was just my example name, so GitHub has no such repo under your account. GitHub also reports "Invalid username or token" when a repo doesn't exist.
2. Because `cd lab-repo` failed, the next commands ran in the wrong place. Your home folder (`~`) is already a git repo, so `git add` and `git commit` made a commit there containing `Downloads/Lab-4/...`, including that `original` folder. Nothing has been pushed yet.

Step 1: undo the stray commit

```
bash
```

```
cd ~
```

```
git log --oneline -3
```

If the top commit is `ecef256 Lab 4: Lava Escape vibe coding`, undo it. Your files stay untouched, only the commit is removed:

```
bash
```

```
git reset HEAD~1
```

If `git log` shows only that one commit, or `git reset` errors, paste the output here before doing anything else.

Step 2: create the real repo (browser)

1. On github.com, click New repository. Pick a name like `pes-lab-repo` and click Create repository. If your course already told you a lab repo name, use that one instead.
2. Create a login token, since GitHub rejects your account password in the terminal. Go to Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic). Tick the repo checkbox, generate it, and copy the token. It's only shown once.

Step 3: push from a clean folder

Replace `YOUR_REPO_NAME` with the real repo name, with no angle brackets:

```
bash
cd ~/Downloads
git clone https://github.com/HarshKalekar/YOUR_REPO_NAME.git
cd YOUR_REPO_NAME
mkdir -p Lab-4
cp -r ../Lab-4/main.py ../Lab-4/requirements.txt ../Lab-4/README.md
../Lab-4/game ../Lab-4/before.mov ../Lab-4/after.mov Lab-4/
rm -rf Lab-4/game/__pycache__

ls Lab-4
```

This copies the files one by one, so the stray `original` folder doesn't come along. Your `chat_history.pdf` isn't in that list because I don't know where you saved it. Copy it in too, for example:

```
bash

cp ~/Downloads/chat_history.pdf Lab-4/
```

Then:

```
bash
git add Lab-4
git status
git commit -m "Lab 4: Lava Escape vibe coding"
```

```
git push
```

When it asks for a username, type `HarshKalekar`. When it asks for a password, paste the token. The terminal shows nothing while you paste, which is normal. Press Enter.

If the push complains about the branch name, run `git push -u origin main`.

If videos are too big

GitHub rejects files over 100 MB. Check sizes with `ls -lh Lab-4`. If a `.mov` is over about 90 MB, open it in QuickTime and use Edit → Trim, then export.

Paste the output if anything fails, especially from the `git log` check in Step 1.

should i create a private or public github repo before doing this?

It depends on how your instructor will check it. The handout doesn't say, so here is how I'd decide:

1. If the course told you a repo name or visibility (for example "create a repo called X and share the link"), follow that exactly.
2. If nothing was specified, make it public. The submission needs your instructor or TAs to open the repo and watch the videos, and a public repo works for anyone with the link and no extra setup. There's nothing sensitive in this project: it's a small game, two screen recordings and a chat export.
3. If you prefer private, that works too, but you must add your instructor as a collaborator (repo Settings → Collaborators → Add people), or they'll see a 404 and may mark it as missing. You'd need their GitHub username from the course.

Public is the safer default for grading, since a private repo you forgot to share looks the same as a missing submission.

Checks before you make it public

- Skim your chat history PDF for anything personal. It will contain your terminal screenshots and file paths, including your name from the prompt (`harshkalekar`). That's probably fine, but you should know.
- Never put your personal access token in any file you commit. Only paste it into the terminal password prompt.
- Don't add a README or `.gitignore` when creating the repo on GitHub, so the first clone is clean. The default of leaving everything unchecked is fine.

The push steps are identical either way. The token you create with the repo checkbox works for both public and private repos.

You can also switch visibility later under Settings → Danger Zone → Change visibility, so the choice isn't permanent. If you're unsure, start with public, and tell me which you picked if you hit any errors on the push.

okay what do i name my github repo

First check whether your course specified a name. Look in the course announcement, the lab handout or your LMS for something like "name your repo `<SRN>-SETA-Labs`". If they gave a naming format, use that exactly, because TAs sometimes find submissions by repo name.

If nothing was specified, pick a clear name that works for all your labs, since the handout says to upload to "your Lab repo under the Lab-4 folder". That implies one repo with a folder per lab. Good options:

- `seta-labs`
- `seta-lab-submissions`
- `harsh-kalekar-seta-labs`, if you want your name in it so it's easy to identify

I'd go with `seta-labs`, or add your SRN or roll number if your course uses one. That matches the course repo (SETAPESU26) and works for Lab 1, 2, 3 and so on.

Naming rules: use lowercase letters, numbers and hyphens, with no spaces. Avoid anything like `Lab-4` as the repo name, since Lab-4 is a folder inside the repo.

Then in the push commands, replace `YOUR_REPO_NAME` with the name you chose. For example, with `seta-labs`:

```
bash
cd ~/Downloads
git clone https://github.com/HarshKalekar/seta-labs.git

cd seta-labs
```

Remember to run the "undo the stray commit" step (`cd ~, git log --oneline -3, git reset HEAD~1`) before you push, so your home folder isn't left with that stray commit.

okay ill name it PES1UG24CS183-SE-LAB

That's a good name. It identifies you by SRN, which graders usually look for, and uppercase letters and hyphens are fine on GitHub.

1. Create the repo

On github.com, click New repository:

- Name: `PES1UG24CS183-SE-LAB`
- Visibility: Public (unless your course told you otherwise)
- Leave "Add a README", ".gitignore" and "license" all unchecked
- Click Create repository

2. Undo the stray commit from earlier

Skip this if you already did it:

```
bash
cd ~

git log --oneline -3
```

If the top commit is `Lab 4: Lava Escape vibe coding`, run:

```
bash
```

```
git reset HEAD~1
```

3. Create a token (if you haven't yet)

GitHub Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic). Tick repo, generate it, and copy it right away.

4. Clone, copy files, push

bash

```
cd ~/Downloads
git clone https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB.git
cd PES1UG24CS183-SE-LAB
mkdir -p Lab-4
cp -r ../Lab-4/main.py ../Lab-4/requirements.txt ../Lab-4/README.md
../Lab-4/game ../Lab-4/before.mov ../Lab-4/after.mov Lab-4/
rm -rf Lab-4/game/__pycache__
```

```
ls Lab-4
```

The `ls` should list `main.py`, `game`, `requirements.txt`, `README.md`, `before.mov` and `after.mov`. Add your chat PDF too, adjusting the path to where you saved it:

bash

```
cp ~/Downloads/chat_history.pdf Lab-4/
```

Then:

bash

```
git add Lab-4
git status
git commit -m "Lab 4: Lava Escape vibe coding"

git push
```

When it asks for a username, type `HarshKalekar`. For the password, paste the token. The screen shows nothing while you paste, which is normal. Press Enter.

If the push complains about the branch, run `git push -u origin main`.

5. Check it

Open <https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB> and confirm Lab-4/ has the code, both videos and the chat PDF.

Paste any error output here and I'll help.

```
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % git add
Lab-4
git status
git commit -m "Lab 4: Lava Escape vibe coding"
git push
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
```

```
nothing to commit, working tree clean
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
```

```
nothing to commit, working tree clean
Username for https://github.com:
```

Press Ctrl + C to cancel that login prompt. Nothing was pushed, and there was nothing to push.

What happened

- "Nothing to commit, working tree clean" means Lab-4 in your repo folder is empty, or the copy step didn't work. Git ignores empty folders, so `git add Lab-4` added nothing. The `cp` line probably errored earlier, or got skipped when you pasted the commands.
- "Upstream is gone" is harmless. Your new repo is empty, so `origin/main` doesn't exist yet. The first push with `-u` fixes it.

Check what's there

You're in `~/Downloads/PES1UG24CS183-SE-LAB`. Run:

```
bash
```

```
ls -la Lab-4
```

```
ls ../Lab-4
```

The first shows what you copied, probably nothing. The second shows what's in the source folder, which should list `main.py`, `game`, `before.mov`, `after.mov` and so on.

Copy the files again

If `../Lab-4` has the files, run this from inside the repo folder:

```
bash
```

```
mkdir -p Lab-4
```

```
cp -r ../Lab-4/main.py ../Lab-4/requirements.txt ../Lab-4/README.md  
../Lab-4/game ../Lab-4/before.mov ../Lab-4/after.mov Lab-4/
```

```
rm -rf Lab-4/game/__pycache__
```

```
ls Lab-4
```

Watch for any error such as `No such file or directory`. If it names a missing file (`before.mov` is the likely one, if it's still on your Desktop), copy it from there instead:

```
bash
```

```
cp ~/Desktop/before.mov Lab-4/
```

Once `ls Lab-4` shows the code and both videos, add your chat PDF if you have it, then push:

```
bash
```

```
git add Lab-4
```

```
git status
```

```
git commit -m "Lab 4: Lava Escape vibe coding"
```

```
git push -u origin main
```

`git status` should now list new files under `Lab-4/` in green. If it still says "nothing to commit", paste the output of the two `ls` commands from above and I'll work out what's missing.

When the login prompt appears, the username is `HarshKalekar` and the password is your personal access token, not your account password.

```
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % mkdir -p Lab-4
cp -r ../Lab-4/main.py ../Lab-4/requirements.txt ../Lab-4/README.md ../Lab-4/game
../Lab-4/before.mov ../Lab-4/after.mov Lab-4/
rm -rf Lab-4/game/__pycache__
ls Lab-4
after.mov      game          README.md
before.mov     main.py       requirements.txt
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % cp
~/Desktop/before.mov Lab-4/
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % git add
Lab-4
git status
git commit -m "Lab 4: Lava Escape vibe coding"
git push -u origin main
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
```

```
nothing to commit, working tree clean
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
```

```
nothing to commit, working tree clean
Username for https://github.com:
```

Good news: your files are copied, and you probably don't need to commit again.

`git status` says "working tree clean" even though `Lab-4` has files. That most likely means the files were already committed locally when you ran the earlier block. That block committed, then stopped at the login prompt, which you cancelled. The only missing step is the push, and the login prompt on your screen right now is it.

Do this now

If the `Username for 'https://github.com':` prompt is still showing, don't cancel it. Type:

1. Username: `HarshKalekar`, then Enter
2. Password: paste your personal access token, then Enter. Nothing appears on screen while you paste, which is normal.

If it asks again or fails, see the checks below.

Confirm the commit exists

In a separate Terminal window or tab (or after the prompt finishes), from inside the repo folder, run:

```
bash
cd ~/Downloads/PES1UG24CS183-SE-LAB
git log --oneline -3

git ls-files
```

You should see `Lab 4: Lava Escape vibe coding in the log`, and `git ls-files` should list `Lab-4/main.py`, `Lab-4/game/...`, `Lab-4/before.mov` and `Lab-4/after.mov`. If so, push with:

```
bash

git push -u origin main
```

If the login fails

- "Invalid username or token": the token is wrong, expired, or missing the repo checkbox. Create a new one (Settings, Developer settings, Personal access tokens, Tokens (classic)) with repo ticked, and use that.
- "Repository not found": check that the repo is spelled exactly `PES1UG24CS183-SE-LAB` and exists on your account.
- Files missing from `git ls-files`: paste its output here and I'll work out what's blocking `git add`.

After a successful push, open

`https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB` to confirm Lab-4/ is there. Don't forget to add your chat PDF afterward (copy it into Lab-4/, then `git add Lab-4, git commit -m "Add chat history", git push`).

```
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % git push -u origin main
```

```
Username for 'https://github.com': HarshKalekar
```

```
Password for 'https://HarshKalekar@github.com':
```

```
remote: Invalid username or token. Password authentication is not supported for Git operations.
```

```
fatal: Authentication failed for
```

```
https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB.git/
```

```
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB %
```

That error means GitHub rejected what you pasted as the password. The usual causes are that you typed your normal GitHub password, or that you haven't created a personal access token yet. GitHub only accepts a token in that field. Your repo and commit are fine, and nothing is lost.

1. Create the token

1. Go to `github.com` → click your profile picture → Settings.
2. Scroll to the bottom of the left sidebar and click Developer settings.
3. Click Personal access tokens → Tokens (classic).
4. Click Generate new token → Generate new token (classic).
5. Name it `lab-push`, set expiry to 30 days, and tick the `repo` checkbox.
6. Click Generate token and copy the token right away (it starts with `ghp_`). GitHub only shows it once.

2. Push again

```
bash
```

```
git push -u origin main
```

- Username: `HarshKalekar`
- Password: paste the token. Nothing shows on screen while you paste, so just press Enter.

If it fails again

Your Mac may have cached the old bad password in the keychain. Clear it and retry:

```
bash
git credential-osxkeychain erase <<EOF
host=github.com
protocol=https
EOF

git push -u origin main
```

Then enter the username and token again.

Easier alternative

If tokens keep giving you trouble, install GitHub Desktop (desktop.github.com), sign in through the browser, choose File → Add Local Repository, pick
~/Downloads/PES1UG24CS183-SE-LAB, and click Publish repository. It handles login for you with no token.

Once the push works, open

<https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB> and check that Lab-4/ is there. Then add the chat PDF in a second commit.

```
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB % git
credential-osxkeychain erase <<EOF
host=github.com
protocol=https
EOF
git push -u origin main
Username for 'https://github.com': HarshKalekar
Password for 'https://HarshKalekar@github.com':
remote: Invalid username or token. Password authentication is not supported for Git
operations.
fatal: Authentication failed for
https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB.git/
(venv) (base) harshkalekar@Harshs-MacBook-Air PES1UG24CS183-SE-LAB %
```

That's the same rejection, so GitHub still isn't accepting what's being pasted. Terminal logins are fiddly, so I'd skip them. Here are the options, easiest first.

Option 1: GitHub Desktop (most reliable)

It signs in through your browser, so there's no token to copy.

1. Download it from desktop.github.com, install it, and open it.
2. Click Sign in to GitHub.com. A browser tab opens, you log in, and you authorize it.
3. Choose File → Add Local Repository, then pick `Downloads/PES1UG24CS183-SE-LAB`.
4. Click Publish repository, or Push origin if it shows that instead. Make sure "Keep this code private" matches what you want. Your commit "Lab 4: Lava Escape vibe coding" is already there, so it pushes straight away.

Option 2: GitHub CLI, also a browser login

If you have Homebrew:

```
bash
brew install gh
gh auth login
```

Choose GitHub.com, then HTTPS, then answer Yes to authenticating Git, then Login with a web browser. Paste the one-time code it shows into the browser. Then:

```
bash
cd ~/Downloads/PES1UG24CS183-SE-LAB
git push -u origin main
```

Option 3: fix the token, if you want to stay in Terminal

Common mistakes with tokens:

- You pasted your normal GitHub password instead of the token. The token starts with `ghp_`.
- The token has no repo checkbox ticked.
- The token has a trailing space or got cut off when copied.

- You created a fine-grained token instead of Tokens (classic). Fine-grained tokens need extra repo permissions that are easy to miss.

If you recreate it, copy it with the copy button next to the token and paste into the password prompt.

Last resort: upload in the browser

On your repo page on github.com, click Add file → Upload files, create the folder by naming files `Lab-4/...`, and drag your files in. Browser uploads are limited to 25 MB per file, so large videos may fail, and folders are fiddly. Use this only if the others don't work.

Try Option 1 first. If something goes wrong, tell me which option you used and what the screen says.

desktop.github.com

Yes, that's the right site. Here's the full flow once you're there.

Install

1. Open desktop.github.com in your browser and click Download for macOS. On your MacBook Air (Apple silicon), choose the Apple silicon version if it offers a choice.
2. Open the downloaded zip, drag GitHub Desktop into your Applications folder, and launch it.

Sign in

1. Click Sign in to GitHub.com.
2. Your browser opens. Log in as `HarshKalekar` and click Authorize desktop.
3. Back in the app, click Finish on the setup screen (the default name and email are fine).

Add your local repo

1. Choose File → Add Local Repository...

2. Click Choose... and select Downloads → PES1UG24CS183-SE-LAB, then click Add Repository.
3. If it says "this directory does not appear to be a Git repository", you picked the wrong folder. It must be the one that contains the hidden .git folder and Lab-4, which is PES1UG24CS183-SE-LAB itself, not Lab-4.

Push

At the top you should see a button saying Push origin (or Publish branch). Click it. Your earlier commit, "Lab 4: Lava Escape vibe coding", is already waiting, so it uploads straight away.

Check

Open <https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB> and confirm Lab-4/ shows the code and both videos.

Then add the chat PDF

1. Copy your chat PDF into Downloads/PES1UG24CS183-SE-LAB/Lab-4/ (Finder works fine for this).
2. In GitHub Desktop, the file appears in the Changes list. Type a summary like Add chat history and click Commit to main.
3. Click Push origin again.

If anything looks off, such as a different button name or an error message, tell me what it says.

Images hidden when shared

That's the right screen. The repo loaded, and "No local changes" is expected because your files are already committed. Only the upload is left.

Click Publish branch

Use the blue button in the middle of the screen, or the Publish branch button at the top. Either does the same thing. It pushes your main branch to

HarshKalekar/PES1UG24CS183-SE-LAB. If a dialog pops up asking about the repository, leave the defaults and confirm.

You can check first that your commit is there: click the History tab and look for "Lab 4: Lava Escape vibe coding". If you see it, the files are included.

Verify

When it finishes, the button changes to "Fetch origin". Click View on GitHub (bottom right of your screenshot) to open the repo in your browser. You should see a Lab-4 folder containing `main.py`, `game/`, `requirements.txt`, `README.md`, `before.mov` and `after.mov`.

Add the chat PDF

1. Export this chat as a PDF (Cmd + P, then Save as PDF).
2. Copy it into `Downloads/PES1UG24CS183-SE-LAB/Lab-4/`. Show in Finder takes you to the repo folder.
3. Back in GitHub Desktop, the file appears under Changes. Type `Add chat history` in the Summary box, click Commit to main, then Push origin.

Tell me what you see after you click Publish, or if you hit any error.

Images hidden when shared

The push worked. Your repo `HarshKalekar/PES1UG24CS183-SE-LAB` is public and has the commit "Lab 4: Lava Escape vibe coding" with a Lab-4 folder. Python shows as 100% of the languages, so the game code is in there.

1. Check the folder contents

Click the Lab-4 folder in your repo. You should see:

- `main.py`, `requirements.txt`, `README.md`
- `game/` (with `game_engine.py`, `player.py`, `world.py`, `__init__.py`)
- `before.mov` and `after.mov`

If a video is missing or has a warning about size, tell me. Click each `.mov` once to make sure GitHub shows it as a file with a plausible size, not 0 bytes.

2. Add the chat history PDF

This is the last deliverable.

1. In your browser, scroll to the top of this chat and press `Cmd + P`, then choose `Save as PDF`. Name it `chat_history.pdf`. Scroll through the whole conversation first so everything has loaded, otherwise the PDF may be cut short.
2. Copy it into `Downloads/PES1UG24CS183-SE-LAB/Lab-4/`. In GitHub Desktop, Repository → Show in Finder takes you to the repo folder.
3. In GitHub Desktop, the file appears under Changes. Type `Add chat history` as the summary, click `Commit to main`, then click `Push origin`.

The easiest alternative is on `github.com`: open the `Lab-4` folder, click `Add file` → `Upload files`, drag in the PDF, and click `Commit changes`.

3. Submit

Your repo link is `https://github.com/HarshKalekar/PES1UG24CS183-SE-LAB`. Check the handout or your LMS for where to submit it. Your repo's README area says "Add a README", which is fine. The handout doesn't require a README at the repo root, and the one inside `Lab-4/` already describes the project.

If you open the repo in a private/incognito window and can see the files, you know your instructor will be able to as well. Let me know if the PDF export gives you any trouble.

This is the read-only copy that you shared of your chat.